Diffusion Models: From Adding Noise to Generating Images
Understanding Denoising-Based Generation through DDPM and LDM

Cao Quang Hung – 24125010 Nguyen Trong Van Viet – 24125023
Huynh Chi Ton – 24125020
University of Science (HCMUS), VNU-HCM
CS420 – Artificial Intelligence

Abstract

How can learning to remove noise enable a model to generate new images? We study this question through two papers. Denoising Diffusion Probabilistic Models (DDPM) corrupt images with Gaussian noise, train a network to predict that noise, and generate by denoising pure noise over many small steps. Latent Diffusion Models (LDM) run the same process in the compressed latent space of an autoencoder, which makes each step cheaper, and condition it on text or other inputs through cross-attention. We derive the forward process, DDPM's variational objective, and its simplified noise-prediction loss, and illustrate forward noising with a worked one-dimensional calculation and a five-level image sequence. With pretrained checkpoints and no training of our own, we compare pixel and latent generators on LSUN Churches (50 DDIM steps, one T4 GPU). The latent model needs a median 2.44 *s per image versus* 6.10 *s, but its peak tensor allocation is higher,* 1.68 *versus* 0.75 *GiB. Saved trajectories for five ImageNet classes show structure emerging as noise is removed, and a small Stable Diffusion test shows that a prompt edit can change an object's color without reversing its left–right position. We relate these observations to published results and to limits set by training data, copyright, and bias.*

1. Introduction

Generating a realistic image requires a model of the whole distribution of natural images, including shape, texture, lighting, and spatial layout. Diffusion models approach this task from an unusual direction. They first destroy image structure by gradually adding Gaussian noise, and then learn to undo the corruption [8, 17]. Latent diffusion extends the idea to compressed representations and underlies text-to-image systems such as Stable Diffusion [14, 15].

Learning objective. Explain forward noising and reverse denoising, including why latent diffusion reduces computational demands.

Guiding question. How can learning to remove noise enable a model to generate new images?

From denoising to generation. A denoiser repairs a given corrupted image, whereas a generator must produce an image that never existed. A bridge exists because the corruption is a known Gaussian process: it supplies training pairs at every noise level, and a network trained on them learns statistical structure shared across many images. Running the network from pure noise then yields a new sample, provided that generation is split into many small steps. A single network trained with squared error to map pure noise to an image would output the conditional mean, approximately the average training image, because x_T carries almost no information about x_0 (Appendix C.4); small stochastic reverse steps avoid this.

Papers and scope. We center the report on two papers. DDPM [8] makes the 2015 diffusion framework [17] practical by training a network to predict the added noise, but it denoises at full image resolution, so every sampling step is expensive. LDM [15] runs the process in the latent space of a separately trained autoencoder and adds cross-attention so that the denoiser can be conditioned on text and other inputs.

Papers versus our own work. From the papers, we synthesize the forward/reverse construction, DDPM's objective and score interpretation, and LDM's two-stage design; published numbers are attributed to their sources (Section 5.4). Our own material is a worked one-dimensional forward step, a five-level noising sequence of a synthetic image, and experiments with pretrained checkpoints: a timing and memory comparison of pixel and latent generators on LSUN Churches, denoising trajectories for five ImageNet classes, and a small prompt-control test with Stable Diffusion v1.4. These experiments use few samples and measure neither FID nor bias, so they are illustrations rather than benchmarks.

Organization. Section 2 gives background, Section 3 develops DDPM with two worked examples, and Section 4 explains LDM. Section 5 reports our experiments and pub-

lished evidence and discusses limits from training data, copyright, and bias; Section 6 concludes.

2. Background and Paper Review

2.1. Generative versus discriminative models

The fundamental distinction between discriminative and generative models lies in what they aim to learn from data. A discriminative model learns to predict a target y given an observation x, typically by estimating $p_\omega(y \mid x)$ or learning a decision boundary between classes. For instance, a cat classifier only needs to identify features that distinguish cats from other objects, without explicitly modeling the distribution of natural images. In contrast, a generative model aims to capture the underlying data distribution, such as $p_\theta(x)$ or the conditional distribution $p_\theta(x \mid y)$. Rather than merely distinguishing categories, it must model the statistical structure and variability of the data, including factors such as shape, texture, lighting, and spatial relationships. This enables the model to generate new samples that resemble the training data, potentially conditioned on labels or text descriptions. Consequently, generative modeling generally requires capturing richer information about the data than is necessary for classification.

2.2. Qualitative Comparison of Generative Models

GANs, autoregressive models, and diffusion models differ in how they learn and sample the data distribution. A *GAN* trains a generator to transform random noise into images while a discriminator learns to distinguish generated images from real ones [5]. Sampling typically takes one generator pass, but training requires balancing the two competing networks. An *autoregressive model* generates an image one pixel or token at a time, predicting each from the previously generated values [21]. This gives an explicit factorization of the image distribution, although sequential generation can be costly. A *diffusion model* learns to reverse gradual noise corruption and generates an image by repeatedly denoising an initially random state [8]. It refines the whole state across multiple steps, requiring repeated network evaluations during sampling. Figure 9 in Appendix E compares the generative mechanisms of these three families.

2.3. Historical context

The foundations of diffusion-based generative modeling were established by Sohl-Dickstein et al. [17], who introduced a probabilistic framework inspired by nonequilibrium thermodynamics. Their approach consists of a forward diffusion process that gradually destroys the structure of the data by adding noise, followed by a learned reverse process that reconstructs samples from noise. By decomposing a complex generative transformation into a sequence of small, tractable transitions, this framework enables likelihood-based training and probabilistic inference. However, generating samples requires many sequential reverse steps, making inference computationally expensive.

An alternative perspective was developed by Song and Ermon [19] through score-based generative modeling. Instead of directly modeling the data probability density, their approach estimates its score function, defined as the gradient of the log-density. To address difficulties in estimating scores when data lie near low-dimensional manifolds, they introduced multiple levels of Gaussian noise and trained a noise-conditional score network. Samples are then generated using annealed Langevin dynamics, which progressively moves samples toward high-density regions as the noise level decreases.

Ho et al. [8] subsequently introduced Denoising Diffusion Probabilistic Models (DDPM), establishing a connection between diffusion probabilistic modeling and denoising score matching. Their formulation parameterizes the reverse process through a neural network that predicts the Gaussian noise added at each diffusion timestep.

Despite their strong generative performance, conventional diffusion models operate directly in high-dimensional pixel space, resulting in substantial training and sampling costs. To address this limitation, Rombach et al. [15] proposed Latent Diffusion Models (LDM). This architecture incorporates cross-attention mechanisms to support flexible conditioning, including text-guided image generation. LDM builds this latent space with a variational autoencoder, reviewed next.

2.4. Variational autoencoders

A variational autoencoder (VAE) [11] is a latent-variable generative model. It assumes that an image $x_0 \in \mathbb{R}^D$ is produced from a latent vector $\tilde{z}$ drawn from a simple prior:

$$p(\tilde{z}) = \mathcal{N}(0, \mathbf{I}), \qquad x_0 \sim p_\psi(x_0 \mid \tilde{z}), \tag{1}$$

where the *decoder* p_ψ is a neural network with parameters ψ. The likelihood $p_\psi(x_0) = \int p(\tilde{z})\, p_\psi(x_0 \mid \tilde{z})\, d\tilde{z}$ is intractable, so an *encoder* with parameters ϕ approximates the posterior over latents:

$$\begin{aligned} q_\phi(\tilde{z} \mid x_0) &= \mathcal{N}\big(\mu_\phi(x_0), \mathrm{diag}(\sigma_\phi^2(x_0))\big)\,, \\ \tilde{z} &= \mu_\phi(x_0) + \sigma_\phi(x_0) \odot \xi, \quad \xi \sim \mathcal{N}(0, \mathbf{I}). \end{aligned} \tag{2}$$

The encoder outputs latent means μ_ϕ and positive standard deviations σ_ϕ; diag forms a diagonal covariance and $\odot$ denotes elementwise multiplication. This reparameterization places the randomness in ξ, so gradients flow through the sampled latent. Encoder and decoder are trained jointly by minimizing the negative evidence lower bound (ELBO):

$$\begin{aligned} \mathcal{J}_{\mathrm{VAE}} &= \mathbb{E}_{q_\phi(\tilde{z}|x_0)}[-\log p_\psi(x_0 \mid \tilde{z})] \\ &\quad + \mathrm{D}_{\mathrm{KL}}(q_\phi(\tilde{z} \mid x_0) \,\|\, \mathcal{N}(0, \mathbf{I})) \;\geq\; -\log p_\psi(x_0). \end{aligned} \tag{3}$$

The first term rewards reconstruction; the KL term pulls every encoding toward the prior, so that decoding a fresh draw $\tilde{z} \sim \mathcal{N}(0, \mathbf{I})$ generates a new image. With a Gaussian decoder, the reconstruction term is a pixel-wise squared error; such pixel losses tend to average over plausible fine details and yield blurry images [15, Sec. 3.1]. The diffusion bound of Section 3.3 follows the same variational idea, with the noisy states $x_{1:T}$ as latents and a fixed, rather than learned, encoder q.

3. Method: DDPM and the Diffusion Framework

We first introduce the forward/reverse construction established by Sohl-Dickstein et al. [17], then focus on DDPM's variational objective, noise-prediction parameterization, simplified loss, and sampling [8]. We use DDPM's $x_t, \alpha_t, \bar{\alpha}_t$ notation throughout. The original 2015 training bound is retained in Appendix C.5.

3.1. Forward process, Gaussian noise, and schedule

A clean image $x_0 \in \mathbb{R}^D$ is gradually corrupted by a Gaussian Markov chain. The Gaussian branch of the 2015 model uses [17, Sec. 2.1, Table C.1]

$$q(x_t \mid x_{t-1}) = \mathcal{N}\Big(\sqrt{1-\beta_t}\, x_{t-1},\, \beta_t \mathbf{I}\Big), \tag{4}$$

$$x_t = \sqrt{1-\beta_t}\, x_{t-1} + \sqrt{\beta_t}\, \varepsilon^{(t)},$$

$$\varepsilon^{(t)} \sim \mathcal{N}(0, \mathbf{I}), \qquad \alpha_t = 1 - \beta_t, \quad \bar{\alpha}_t = \prod_{j=1}^{t} \alpha_j.$$

Here D counts normalized image coordinates, $\mathbf{I}$ is the identity matrix, and $t \in \{1, \dots, T\}$ is a step index, not physical time. The innovations $\varepsilon^{(t)}$ are independent across steps and coordinates. The dimensionless variance $\beta_t \in (0, 1)$ simultaneously attenuates the previous signal by $\sqrt{1-\beta_t}$ and sets the new noise standard deviation $\sqrt{\beta_t}$. The schedule $(\beta_1, \dots, \beta_T)$ controls how rapidly structure is lost.

The standard Gaussian is stationary under this kernel: if a state has covariance $\mathbf{I}$ and is standard normal, the next state's covariance is $(1-\beta_t)\mathbf{I} + \beta_t \mathbf{I} = \mathbf{I}$. Repeated small steps can therefore transform a complex image distribution toward a simple Gaussian prior. In the 2015 Gaussian experiments, β_1 is fixed to a small value while $\beta_2, \dots, \beta_T$ are learned through the variational bound; the paper also notes that a fixed schedule can work comparably [17, Sec. 2.4.1]. The Gaussian kernel's form is known even when its rates are optimized.

Repeated Gaussian composition gives, with $\bar{\alpha}_0 = 1$,

$$x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1-\bar{\alpha}_t}\, \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, \mathbf{I}). \tag{5}$$

This is the same Gaussian marginal later used by DDPM, written in its notation; it is not a new forward-process invention. The aggregate noise ε differs from an individual innovation $\varepsilon^{(t)}$. For small $\bar{\alpha}_T$, the terminal distribution approaches standard normal; a finite schedule need not make that equality exact.

3.2. Reverse model and sampling

Let $p_{\text{data}}(x_0)$ denote the unknown data distribution. The original diffusion model learns a finite reverse Markov chain, starting from a simple Gaussian prior $p(x_T) = \mathcal{N}(0, \mathbf{I})$ [17, Sec. 2.2]:

$$p_\theta(x_{0:T}) = p(x_T) \prod_{t=1}^{T} p_\theta(x_{t-1} \mid x_t),$$

$$p_\theta(x_{t-1} \mid x_t) = \mathcal{N}(\mu_\theta(x_t, t), \Sigma_\theta(x_t, t)). \tag{6}$$

Although the true reverse conditional depends on the unknown data distribution and is not necessarily Gaussian, sufficiently small diffusion steps motivate approximating each reverse transition with a Gaussian distribution.

For learned intermediate transitions, neural functions f_μ and f_Σ parameterize the mean $\mu_\theta(x_t, t)$ and covariance $\Sigma_\theta(x_t, t)$, respectively. The mean specifies the expected previous state, while the covariance captures uncertainty in the reverse transition. Unlike DDPM's noise-prediction parameterization (Section 3.3), the original model directly estimates these reverse-distribution parameters.

To generate a sample, the model draws $x_T \sim \mathcal{N}(0, \mathbf{I})$ and iteratively applies the reverse transitions:

$$x_{t-1} = \mu_\theta(x_t, t) + \Sigma_\theta(x_t, t)^{1/2} u_t, \quad u_t \sim \mathcal{N}(0, \mathbf{I}). \tag{7}$$

For diagonal covariance, the matrix square root corresponds to elementwise standard deviations. The original formulation additionally specifies a boundary convention for the final transition at $t = 1$ [17, App. B.2], detailed in Appendix C.5. Through these successive stochastic transitions, the model transforms Gaussian noise into samples approximating p_{data}, without requiring a clean input image.

Figure 1 shows the architecture of both flows: a Gaussian kernel corrupts each forward state, while a neural model parameterizes the reverse distribution from which the previous state is sampled. Appendix F gives step-by-step pseudocode for forward noising and the 2015 Gaussian reverse sampler (Algorithms 1 and 2).

3.3. DDPM: From Variational Training to Noise and Score Prediction

Variational training and Gaussian posterior targets. DDPM inherits the forward/reverse construction above and its likelihood-based training [8]. Since computing $p_\theta(x_0)$ requires marginalizing over all intermediate states, training minimizes a variational upper bound on the negative log likelihood [8, Eq. 5]:

$$L_{\text{VLB}} = L_T + \sum_{t=2}^{T} L_{t-1} + L_0. \tag{8}$$

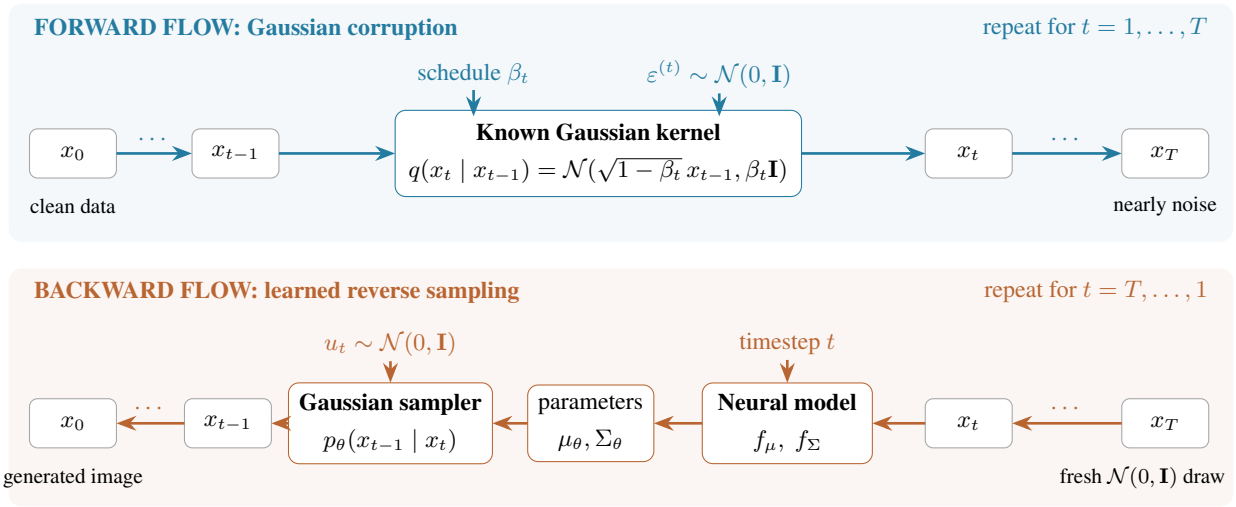


Figure 1. Architecture of the Gaussian diffusion flows [8, 17]. Forward steps (blue) use the noise schedule and Gaussian innovations. Backward steps (orange) start from fresh noise; a neural model predicts reverse-distribution parameters and a Gaussian sampler produces the next state. The learned intermediate steps are shown; the final-step boundary is discussed in the text.

Here, L_T matches the terminal forward distribution to the Gaussian prior, L_0 represents the final reconstruction likelihood, and each intermediate term L_{t-1} measures the KL divergence between the learned reverse transition $p_\theta(x_{t-1} \mid x_t)$ and the forward posterior $q(x_{t-1} \mid x_t, x_0)$.

Although the unconditional reverse distribution is unknown, conditioning on the clean training image x_0 gives a tractable Gaussian posterior:

$$q(x_{t-1} \mid x_t, x_0) = \mathcal{N}(\tilde{\mu}_t, \tilde{\beta}_t \mathbf{I}), \qquad t > 1. \tag{9}$$

This provides an explicit training target for the reverse distribution without requiring the data density itself. The original diffusion framework learns both the reverse mean and covariance. DDPM simplifies this formulation by fixing the forward noise schedule and choosing a fixed reverse variance $\Sigma_\theta = \sigma_t^2 \mathbf{I}$.

Under this assumption, the intermediate Gaussian KL reduces, up to a constant independent of θ, to mean matching [8, Eq. 8]:

$$L_{t-1} = \mathbb{E}_q \left[\frac{\|\tilde{\mu}_t - \mu_\theta(x_t, t)\|_2^2}{2\sigma_t^2} \right] + C. \tag{10}$$

Thus, minimizing the variational bound requires learning the posterior mean of each reverse transition. Appendix C.5 gives the full Gaussian KL, including the covariance terms that become constant under this assumption.

From mean prediction to noise prediction. Rather than predicting this mean directly, DDPM exploits the closed-form marginal of Eq. (5) (Appendix C.3). Substituting the expression for x_0 into the Gaussian posterior mean gives [8, Eqs. 10–11]:

$$\tilde{\mu}_t = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}} \varepsilon \right), \tag{11}$$

$$\mu_\theta(x_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(x_t - \frac{\beta_t}{\sqrt{1-\bar{\alpha}_t}} \varepsilon_\theta(x_t, t) \right). \tag{12}$$

The second expression replaces the sampled Gaussian noise with a neural prediction $\varepsilon_\theta(x_t, t)$. Consequently, predicting noise is an alternative parameterization of the reverse mean, rather than a different probabilistic model.

Connection to denoising score matching. This parameterization also connects DDPM to score-based generative modeling (Section 2.3) [19]. The score of the noisy data distribution $q_t(x_t)$ is defined as $\nabla_{x_t} \log q_t(x_t)$, which points toward regions of increasing probability density. Appendix C.2 provides background on scores and Gaussian perturbations.

Because the forward corruption is Gaussian, the conditional score can be written as

$$\nabla_{x_t} \log q(x_t \mid x_0) = -\frac{\varepsilon}{\sqrt{1-\bar{\alpha}_t}}. \tag{13}$$

Taking the conditional expectation over clean images given x_t yields the marginal score:

$$\nabla_{x_t} \log q_t(x_t) = -\frac{\mathbb{E}[\varepsilon \mid x_t]}{\sqrt{1-\bar{\alpha}_t}}. \tag{14}$$

Under squared-error training, the optimal noise predictor estimates $\mathbb{E}[\varepsilon \mid x_t]$. Appendix C.4 derives this MSE

optimum and the conditional-to-marginal score identity; Appendix C.7 explains why the estimate need not recover the particular noise draw of a training example. Therefore, the learned noise predictor implicitly estimates the noisy-data score:

$$s_\theta(x_t, t) := -\frac{\varepsilon_\theta(x_t, t)}{\sqrt{1-\bar{\alpha}_t}} \approx \nabla_{x_t} \log q_t(x_t). \tag{15}$$

Substituting this relation into Eq. (12) gives an equivalent score-based expression:

$$\mu_\theta(x_t, t) = \frac{1}{\sqrt{\alpha_t}} \left(x_t + \beta_t s_\theta(x_t, t)\right). \tag{16}$$

Appendix C.6 derives the corresponding identity for the true reverse conditional mean and clarifies the Gaussian approximation.

Hence, reverse-mean prediction, noise prediction, and score estimation are closely related representations of the same denoising function. This connection explains why DDPM can be interpreted through denoising score matching while retaining its probabilistic formulation.

Weighted and simplified noise objectives. Substituting the noise parameterization into the Gaussian mean-matching loss gives [8, Eq. 12]:

$$L_{t-1} = \mathbb{E}_{x_0,\varepsilon}\left[w_t \|\varepsilon - \varepsilon_\theta(x_t, t)\|_2^2\right] + C, \tag{17}$$

$$w_t = \frac{\beta_t^2}{2\sigma_t^2 \alpha_t (1-\bar{\alpha}_t)}, \qquad t > 1. \tag{18}$$

This weighted noise-prediction objective corresponds to the intermediate terms of the variational bound; at each noise level it is a rescaled score-matching loss (Appendix C.4).

DDPM further simplifies training by dropping the timestep-dependent weights:

$$L_{\text{simple}} = \mathbb{E}_{t,x_0,\varepsilon}\left[\|\varepsilon - \varepsilon_\theta(x_t, t)\|_2^2\right]. \tag{19}$$

Here, t is sampled uniformly, and each noisy input x_t is constructed directly from a clean training image and sampled Gaussian noise.

Removing w_t changes the relative emphasis placed on different noise levels. Ho et al. report improved sample quality under this simplified objective, while the original variational objective provides better likelihood-based code-lengths [8, Secs. 3.4, 4.1–4.2], an empirical trade-off rather than a general guarantee.

Note that L_{simple} is a surrogate, not the complete bound: it omits the decoder term L_0 and regresses noise at all timesteps, including $t = 1$.

Reverse sampling. Once trained, the same noise predictor determines the reverse sampling process. DDPM uses a fixed reverse variance, with two choices investigated by Ho et al. [8, Sec. 3.2]:

$$\sigma_t^2 = \beta_t \quad \text{or} \quad \sigma_t^2 = \tilde{\beta}_t = \frac{1-\bar{\alpha}_{t-1}}{1-\bar{\alpha}_t}\beta_t, \qquad t > 1. \tag{20}$$

Starting from $x_T \sim \mathcal{N}(0, \mathbf{I})$, the model iteratively samples [8, Algorithm 2]:

$$x_{t-1} = \mu_\theta(x_t, t) + \sigma_t u_t, \quad u_t \sim \mathcal{N}(0, \mathbf{I}), \qquad t > 1. \tag{21}$$

At $t = 1$, the original DDPM sampler sets the added noise to zero and outputs $\mu_\theta(x_1, 1)$. Appendix F gives forward/backward pseudocode and distinguishes this endpoint from the original 2015 sampler. In score form, each step adds a learned density-increasing direction and fresh noise, resembling Langevin dynamics.

3.4. A complete one-dimensional calculation

For one step $t = 1$, choose normalized $x_0 = 0.8$, variance $\beta_1 = 0.1$, and Gaussian realization $\varepsilon = -0.5$. This larger variance makes the arithmetic visible; the image schedule below uses smaller steps. Equation (4) gives

$$\begin{aligned} \alpha_1 &= 1 - 0.1 = 0.9, \\ \sqrt{\alpha_1}\, x_0 &= \sqrt{0.9}\,(0.8) = 0.758947, \\ \sqrt{\beta_1}\, \varepsilon &= \sqrt{0.1}\,(-0.5) = -0.158114, \\ x_1 &= 0.758947 - 0.158114 = \boxed{0.600833}. \end{aligned}$$

Thus $q(x_1 \mid x_0 = 0.8) = \mathcal{N}(0.758947, 0.1)$: its mean is 0.758947, variance 0.1, and standard deviation $\sqrt{0.1} = 0.316228$. Across independent draws, $\mathbb{E}[x_1 \mid x_0] = \sqrt{0.9}\, x_0$ since $\mathbb{E}[\varepsilon] = 0$. The realization 0.600833 differs from that expectation; the conditional variance is β_1.

3.5. Five noise levels on an original small image

We apply Eq. (4) step by step to a synthetic 28×28 image of a house, with foreground $+1$ and background -1. Following DDPM [8, Sec. 4], we use $T = 1000$ and a linear schedule increasing from $\beta_1 = 10^{-4}$ to $\beta_T = 0.02$. Figure 2 shows one sampled trajectory at five timesteps.

In terms of Eq. (5), the signal-to-noise ratio $\bar{\alpha}_t/(1-\bar{\alpha}_t)$ drops from 33.5 at $t = 50$, where the house is clearly visible, to below 0.1 at $t = 500$, where the image is dominated by noise. By $t = T$ almost no signal remains, which justifies starting generation from $\mathcal{N}(0, \mathbf{I})$. This illustrates corruption only.



Figure 2. Forward noising of a synthetic house image along one trajectory. Values are clipped to $[-1, 1]$ for display only.

4. LDM: Latent Space and Efficient Generation

4.1. Why move from pixels to a latent representation?

The diffusion framework supplies a way to generate images, but repeatedly processing a high-resolution pixel grid is expensive. LDM separates learning a useful image representation from learning its generative distribution [15, Secs. 3.1–3.2]. Its first stage compresses images; its second stage applies diffusion to the resulting spatial tensors.

For RGB input $x_0 \in \mathbb{R}^{H\times W\times 3}$, let $\tilde{z} \in \mathbb{R}^{h\times w\times c}$, where $h = H/f$, $w = W/f$, $f \geq 1$ is the spatial downsampling factor, and c is the latent channel count. For example, an LDM configuration with $f = 4, c = 3$ maps $256\times256\times3$ to $64\times64\times3$ [15, Table 8]. This reduces the number of spatial positions by 16, but the end-to-end runtime and memory depend on more than the state shape.

4.2. Perceptual image compression

LDM's first stage is an autoencoder with encoder $\mathcal{E}_\phi$ and decoder D_ψ. It starts from the VAE of Section 2.4, but its only job is to compress images [15, Sec. 3.1]:

$$\tilde{z} = \mathcal{E}_\phi(x_0) \in \mathbb{R}^{h\times w\times c}, \qquad \hat{x}_0 = D_\psi(\tilde{z}). \tag{22}$$

Unlike the flat vector of Eq. (1), $\tilde{z}$ keeps a spatial layout that a convolutional U-Net [16] can process. In the KL variant below, $\mathcal{E}_\phi$ samples $\tilde{z}$ with the reparameterization of Eq. (2); this posterior noise ξ differs from the diffusion noise added in the second stage.

Training objective. Instead of the pixel likelihood in Eq. (3), the autoencoder combines three ingredients [15, Sec. 3.1]. A *perceptual loss* [25] compares deep-network features, so a reconstruction must look like the input rather than match every pixel. A *patch-based adversarial loss* [4, 10] lets a discriminator judge local patches, pushing textures toward realism and avoiding the blur of pixel-only losses. A light *regularizer* prevents latents with arbitrarily high variance.

Two regularizers. *KL-reg.* adds a weak KL penalty toward $\mathcal{N}(0, \mathbf{I})$, as in Eq. (3) but with a small weight. *VQ-reg.* uses a vector-quantization layer [22]: each latent vector $\tilde{z}_{ij} \in \mathbb{R}^c$ is replaced by the nearest entry of a learned codebook $\mathcal{Z} \subset \mathbb{R}^c$,

$$\mathrm{VQ}(\tilde{z})_{ij} = \arg\min_{e\in\mathcal{Z}} \|\tilde{z}_{ij} - e\|_2. \tag{23}$$

LDM places this layer *inside the decoder*: the model is a VQGAN [4] whose quantization step is absorbed by D_ψ [15, Sec. 3.1]. The diffusion model therefore operates on the continuous, pre-quantization latent, and quantization happens only when a latent is decoded.

Why absorb quantization into the decoder? The choice brings four benefits. (i) *Gaussian diffusion stays well defined.* Forward noising (Eq. (24)) and noise regression require a continuous space; Gaussian noise cannot be added to discrete codebook indices without switching to a different, discrete diffusion process. Both variants expose a continuous latent, so one denoiser and sampler design serves both. (ii) *Tolerance to sampling error.* A generated latent need not land exactly on codebook vectors; the decoder snaps each position to its nearest code, projecting small residual denoising errors back onto latents the decoder was trained on. This is our interpretation of the design, not a claim tested in the paper. (iii) *Spatial structure and mild compression.* VQGAN-style pipelines model token indices autoregressively in an arbitrary 1D order, ignoring much of the 2D structure of the latent. A U-Net models the 2D latent directly, so LDM can use mild factors $f = 4$–8 and keep reconstructions faithful [15, Sec. 3.1]. (iv) *Bounded latent scale.* The codebook bounds latent magnitudes without tuning a KL weight, whereas KL latents are rescaled by the fixed factor a of Section 4.3 [15]. Empirically, VQ-regularized LDMs sometimes reach better sample quality even though their reconstructions slightly trail the KL autoencoders [15, Sec. 4, Table 8]. Our experiments use both: the ImageNet model is VQ-regularized with $f = 4$, and the LSUN Churches model is KL-regularized with $f = 8$ (Table 1).

Diffusion replaces the VAE prior. Neither regularizer makes clean latents standard normal, so LDM learns their distribution with diffusion (Section 4.3) instead of decoding $\tilde{z} \sim \mathcal{N}(0, \mathbf{I})$. Reconstruction quality limits the whole generator: excessive compression may erase fine text or small objects.

4.3. Diffusion training and sampling in latent space

The trained autoencoder is frozen while the latent denoiser is trained. Let $z_0 = a\tilde{z}$, with optional fixed scale $a > 0$ to match latent magnitudes to the noise schedule; $a = 1$ means no rescaling. Diffusion acts on $d_z = hwc$ latent coordinates:

$$z_t = \sqrt{\bar{\alpha}_t}\, z_0 + \sqrt{1 - \bar{\alpha}_t}\, \varepsilon_z, \quad \varepsilon_z \sim \mathcal{N}(0, \mathbf{I}_{d_z}), \tag{24}$$

$$L_{\mathrm{LDM}} = \mathbb{E}_{x_0, y, t, \xi, \varepsilon_z} \left[\|\varepsilon_z - \varepsilon_\theta(z_t, t, \tau_\eta(y))\|_2^2 \right]. \tag{25}$$

The condition y is encoded by τ_η with parameters η (Section 4.4). Eq. (25) follows LDM [15, Eq. 3] with the stochastic encoding made explicit.

For generation, start with latent Gaussian noise and repeatedly apply a reverse sampler with the conditional predictor. Samplers such as DDIM [18] reduce the number of steps, complementing the cheaper latent steps. Decode the final latent once as $D_\psi(z_0/a)$, undoing the training scale.

4.4. Conditioning and the complete pipeline

LDM is not specific to text. It conditions the denoiser on an input y from an arbitrary modality through a domain-specific encoder τ_η [15, Sec. 3.3].

4.4.1. Conditioning through cross-attention

The encoder maps the condition to a sequence of m token embeddings, $C = \tau_\eta(y) \in \mathbb{R}^{m \times d_y}$, and cross-attention layers inserted into the U-Net at several resolutions inject C into the denoiser $\varepsilon_\theta(z_t, t, \tau_\eta(y))$ [15, Sec. 3.3]. For text, LDM trains a transformer τ_η jointly with the denoiser, whereas Stable Diffusion v1.4 uses a frozen CLIP ViT-L/14 text encoder [13, 14]. We use text as the running example below.

At one layer, flatten the U-Net feature map into $F \in \mathbb{R}^{n \times d_u}$, one row per spatial position. A cross-attention head [23] computes

$$Q = FW_Q, \quad K = CW_K, \quad V = CW_V, \tag{26}$$

$$A = \mathrm{softmax}\left(QK^\top / \sqrt{d_a}\right), \qquad O = AV, \tag{27}$$

with learned projections W_Q, W_K, W_V (Figure 3). *Queries come from the image; keys and values come from the condition.* Row i of $A \in \mathbb{R}^{n \times m}$ is a probability distribution over tokens: it measures how well the query of position i matches each token key, with the $\sqrt{d_a}$ scaling keeping the softmax from saturating. Row i of O is the corresponding weighted average of token values; it is projected back to d_u channels and added to the feature map through a residual connection. Each location therefore retrieves the words relevant to it; a region forming a bus can draw on the tokens "red" and "bus". Several heads run in parallel and their outputs are concatenated [23]. Because A is recomputed from the noisy features at every layer and step, the prompt steers the denoising predictions throughout sampling; it is not pasted onto the decoded image.

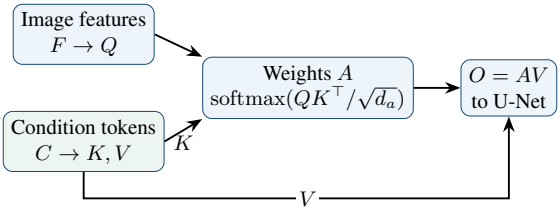


Figure 3. Single-head cross-attention, based on [15, 23]. Image positions query the condition tokens; the output is added back to the U-Net features.

Cross-attention mixing is soft: an attribute can attach to the wrong object, or a token can be neglected [3]; Section 5.5 observes such failures. Matrix shapes and a toy calculation are in Appendix D.1.

Classifier-free guidance can strengthen conditional influence by using $\varepsilon_u + s(\varepsilon_c - \varepsilon_u)$, where $\varepsilon_u, \varepsilon_c$ are unconditional and conditional predictions and $s \geq 0$ is the guidance scale [7]; $s = 1$ is ordinary conditional sampling. Stronger guidance trades diversity for fidelity but does not guarantee correct counting, spatial relations, or attribute binding (Appendix D.2).

4.4.2. A modality-agnostic pipeline

Only τ_η depends on the modality; the autoencoder, the U-Net, and the attention layers are shared [15, Sec. 3.3]. For *text*, τ_η is a transformer or pretrained language encoder with one token per word piece. For a *class label*, a learned embedding yields a single token ($m = 1$), as in our ImageNet LDM. For a *layout*, bounding boxes with class labels are tokenized and encoded. *Spatially aligned* inputs, such as semantic maps, low-resolution images, or inpainting masks, can instead be concatenated with z_t along the channel axis. With *no condition*, as in our LSUN Churches model, τ_η is dropped. LDM evaluates all of these settings [15, Secs. 4.3–4.5].

Figure 4 assembles the pipeline. *Training*: train the autoencoder once and freeze it; encode an image, add noise to its latent, encode the paired condition with τ_η, and train the conditional U-Net with Eq. (25). *Generation*: draw $z_T \sim \mathcal{N}(0, \mathbf{I})$, encode y once, run the sampler with the conditional (and, with CFG, unconditional) predictions, and decode once; no real image or encoder is needed. One autoencoder serves many conditions [15, Sec. 1].

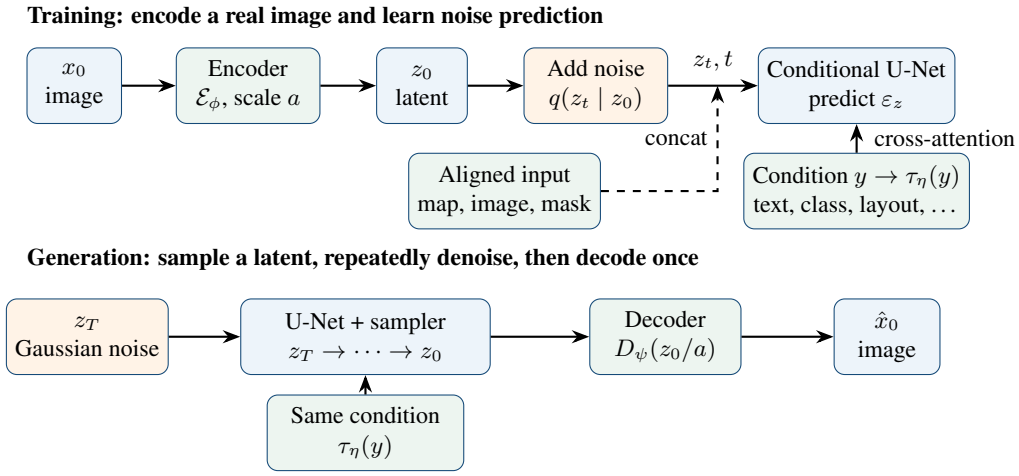


Figure 4. Original latent diffusion pipeline, based on [15, Fig. 3 and Sec. 3]. The autoencoder is learned first and frozen for diffusion training; its decoder runs after the denoising loop. A modality-specific encoder τ_η feeds the condition through cross-attention, while spatially aligned inputs can instead be concatenated with the noisy latent (dashed).

5. Results and Discussion

5.1. Experimental configuration

Table 1 summarizes the three pretrained inference runs. LSUN Churches is our pixel-versus-latent comparison. ImageNet shows intermediate denoising states for both representations. Stable Diffusion v1.4 adds a small text-conditioning demonstration [14]. Appendix B gives the run records.

Table 1. Colab settings. All runs used a Tesla T4, PyTorch `2.14.1+cu130`, FP16 or FP32 as listed, batch size one, and 50 DDIM updates with $\eta = 0$.

Setting	LSUN Churches	ImageNet	Text prompt demo
Pixel checkpoint	Google `ddpm-ema-church-256`	OpenAI ADM `256x256_diffusion.pt`	—
Latent checkpoint	CompVis LDM-KL-8 (`lsun_churches.zip`)	CompVis `cin256-v2` (VQ, $f = 4$)	CompVis Stable Diffusion v1.4
RGB output	256×256	256×256	512×512
Conditioning	Unconditional	Class ID. LDM uses CFG $s = 3.0$	Text. CFG $s = 7.5$
Precision	FP32 for both	ADM FP16 U-Net torso and FP32 states. LDM FP32	FP16
Seeds and samples	2026–2029, four timed images per checkpoint	2026, one image per class for each checkpoint across five classes	2026–2029, four prompts per seed (16 images)
Recorded evidence	Latency, peak allocation, and images	Final images and forward and reverse states	Final images and visual prompt checks

For LSUN, we timed each sample from the initial noise draw to the final RGB tensor with GPU synchronization. The timed interval includes the latent decoder and excludes checkpoint loading, one warm-up image per model, and PNG export. For ImageNet, we saved intermediate reverse states and ran a separate Gaussian forward pass from each final state. We decoded latent states for display.

For the text demo, the baseline asks for a bus and a car. The reference asks for a red bus to the left of a blue car. Two edits change only *red* to *green* or *left* to *right*. Within each seed, all four prompts start from a copy of the same Gaussian latent and follow the conditioning pipeline of Section 4.4. We visually checked the two vehicles, the requested colors, and the relative position. The baseline specifies only which vehicles to show.

5.2. Pixel versus latent diffusion on LSUN Churches

Table 2 compares speed, memory, and visual quality on the same task. We inspected recognizable church features, outlines, and fine detail across all four timed samples per model.

Table 2. Comparison on LSUN Churches under the settings in Table 1. Latency is the median of four images. Peak allocation is PyTorch's peak CUDA tensor allocation.

Criterion	Pixel DDPM	LDM-KL-8
Denoising state	$256 \times 256 \times 3$	$32 \times 32 \times 4$
Latency (s/image)	6.10	2.44
Peak allocation (GiB)	0.75	1.68
Visual quality	Clear steeple in seed 2028. Darker seed 2029 obscures details.	Clear facades in seeds 2027–2029. Seed 2026 shows a distant complex.



Figure 5. All eight timed LSUN Churches samples, grouped by seed. Each model draws its initial noise in its own state space.

The latent pipeline achieved a $2.50\times$ speedup by median latency and used more peak tensor memory (Table 2). Both produced recognizable church imagery (Figure 5). The sample count, checkpoints, and implementations limit conclusions to the tested pipelines.

5.3. ImageNet samples and denoising trajectories

In the ImageNet run, ADM denoised an RGB state and CompVis denoised a $64 \times 64 \times 3$ latent. Figure 6 shows all ten outputs for golden retriever, tabby cat, school bus, monarch butterfly, and volcano (Table 1). We assess recognizable class features and coherence of shape.

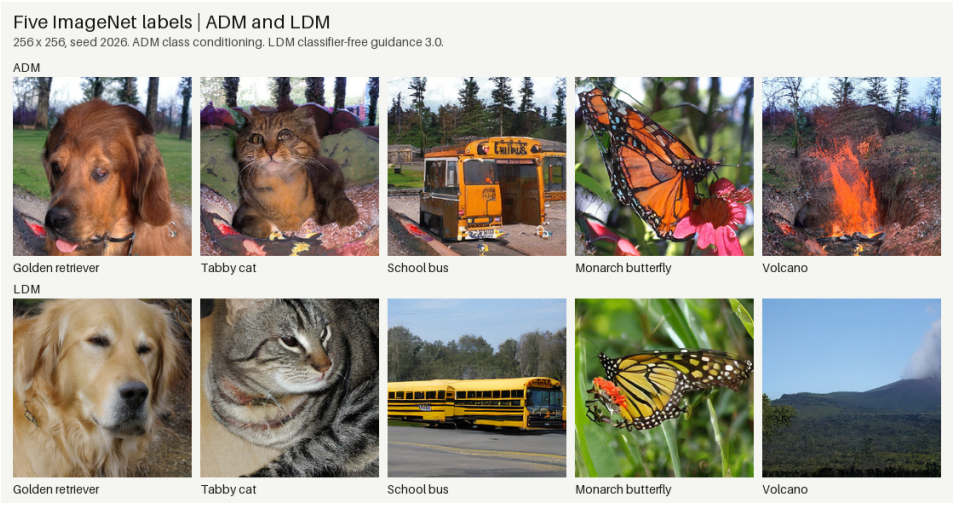


Figure 6. Our ImageNet samples for five shared class labels. The top row uses pixel-space ADM and the bottom row uses latent diffusion.

Reverse sampling starts from a Gaussian draw. At each update the denoiser predicts noise, the sampler estimates the clean state, and that estimate helps compute the next state. At update 25, the clean estimates already show the retriever's head and muzzle, while the updated states still carry visible noise (Figure 7). In a separate forward pass, we start from each model's final state and add Gaussian noise using its schedule. The retriever's features gradually disappear. Appendix G shows both directions for all five labels.

Both bus samples show yellow bodies and rows of windows. The volcano appears as a lava scene in ADM and as a slope with a plume in LDM. The different outputs illustrate variation under the same label. One sample per class and different guidance settings limit the comparison to the examples shown.

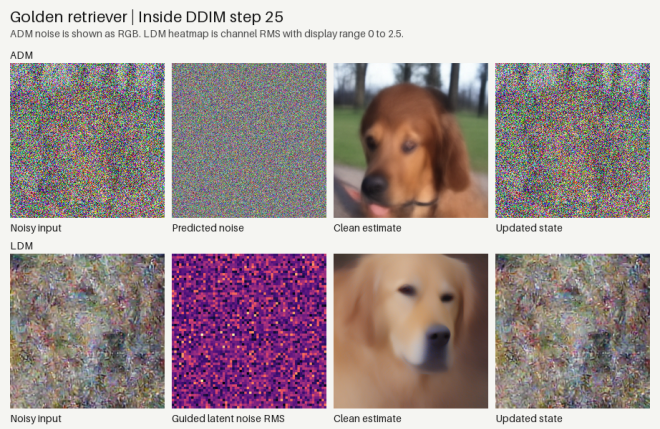


Figure 7. Inside DDIM update 25 for the golden retriever. Read each row from the noisy input through the predicted noise and clean estimate to the updated state. ADM noise is displayed in RGB. The LDM heatmap shows the root mean square of the guided noise across latent channels. The other LDM panels show decoded states.

5.5. Supplementary text prompt results

Changing the requested bus color produced a green bus at all four seeds. None of those images clearly showed the requested blue passenger car. Changing the requested side yielded no clear left-to-right reversal within a seed. At seed 2027, the reference image shows a red bus left of a blue car. The color edit produces two green buses, and the side edit leaves the red bus on the left (Figure 8). The four seeds show how attribute changes can succeed while object identity or spatial relations fail. Appendix H contains all 16 images.



Figure 8. Seed 2027 with the same initial latent. The bus changes to green in the middle image and stays on the left in the rightmost image.

5.6. Training data, copyright, and bias

Training data limits the subjects and variations a model learns. The church checkpoints specialize in church imagery, and ImageNet conditioning uses a fixed class vocabulary. Text-conditioned models also depend on the attributes and relations described in captions. Bianchi et al. [1] found demographic stereotypes in outputs from ordinary prompts.

5.4. Published evidence on quality and speed

For a quantitative comparison within one task, Rombach et al. [15, Table 6] report 0.26 versus 0.97 samples/s at 256^2 and FID@2k 24.74 versus 15.21 after six epochs for pixel LDM-1 and latent LDM-4 (KL with attention), respectively, in Places inpainting. FID compares generated and reference feature distributions [6]. These figures describe the paper's inpainting setup.

DDPM objective ablation. On unconditional CIFAR-10, Ho et al. [8, Tables 1–2] report training-reference FID of 13.51 for the variational objective and 3.17 for the simplified objective with epsilon prediction and fixed isotropic variance. The reported NLL upper bound changes from 3.70 to 3.75 bits per dimension.

The vehicle prompts in our demo do not assess demographic bias. Carlini et al. [2] extracted memorized training images from diffusion models. Data provenance and potential reproduction matter for copyright. The U.S. Copyright Office discusses use of protected works in training under U.S. law [20].

6. Conclusion

Learning to remove noise enables generation by fitting a sequence of reverse conditional distributions. The 2015 diffusion framework supplies forward Gaussian corruption, learned reverse mean/covariance functions, and variational training against tractable posterior targets. Starting from a new Gaussian draw and composing the reverse transitions then produces an image sample. DDPM retains this construction while fixing variance choices, parameterizing the reverse mean through predicted noise, and simplifying the training objective. Its published ablation supports a sample-quality benefit from that objective choice, distinct from likelihood optimization or fewer sampling steps. The score connection explains the parameterization without adding a separate computational stage.

LDM preserves this denoising principle in a learned representation. The ImageNet traces show recognizable structure emerging in both spaces. In our LSUN Churches run, the latent pipeline had lower sampling latency and higher peak tensor allocation. These measurements apply to the tested checkpoints. The supplementary text demonstration shows changes in bus color and persistent failures on object identity and spatial layout.

References

[1] Federico Bianchi, Pratyusha Kalluri, Esin Durmus, Faisal Ladhak, Myra Cheng, Debora Nozza, Tatsunori Hashimoto, Dan Jurafsky, James Zou, and Aylin Caliskan. Easily accessible text-to-image generation amplifies demographic stereotypes at large scale. In *Proceedings of the ACM Conference on Fairness, Accountability, and Transparency*, 2023.

[2] Nicholas Carlini, Jamie Hayes, Milad Nasr, Matthew Jagielski, Vikash Sehwag, Florian Tramèr, Borja Balle, Daphne Ippolito, and Eric Wallace. Extracting training data from diffusion models. In *32nd USENIX Security Symposium*, 2023.

[3] Hila Chefer, Yuval Alaluf, Yael Vinker, Lior Wolf, and Daniel Cohen-Or. Attend-and-excite: Attention-based semantic guidance for text-to-image diffusion models. *ACM Transactions on Graphics*, 42(4), 2023.

[4] Patrick Esser, Robin Rombach, and Björn Ommer. Taming transformers for high-resolution image synthesis. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 12873–12883, 2021.

[5] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial nets. In *Advances in Neural Information Processing Systems*, 2014.

[6] Martin Heusel, Hubert Ramsauer, Thomas Unterthiner, Bernhard Nessler, and Sepp Hochreiter. GANs trained by a two time-scale update rule converge to a local nash equilibrium. In *Advances in Neural Information Processing Systems*, 2017.

[7] Jonathan Ho and Tim Salimans. Classifier-free diffusion guidance. arXiv:2207.12598, 2022. A short version appeared at a NeurIPS 2021 workshop.

[8] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. In *Advances in Neural Information Processing Systems*, pages 6840–6851, 2020.

[9] IEEE. ICCV 2025 Author Kit, 2025. Official author kit; style and bibliography files distributed with the project.

[10] Phillip Isola, Jun-Yan Zhu, Tinghui Zhou, and Alexei A. Efros. Image-to-image translation with conditional adversarial networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2017.

[11] Diederik P. Kingma and Max Welling. Auto-encoding variational bayes. In *International Conference on Learning Representations*, 2014.

[12] OpenAI. Codex, 2026. AI assistance for drafting, LaTeX, code, and source checking; see Appendix B.

[13] Alec Radford, Jong Wook Kim, Chris Hallacy, Aditya Ramesh, Gabriel Goh, Sandhini Agarwal, Girish Sastry, Amanda Askell, Pamela Mishkin, Jack Clark, Gretchen Krueger, and Ilya Sutskever. Learning transferable visual models from natural language supervision. In *Proceedings of the 38th International Conference on Machine Learning*, pages 8748–8763, 2021.

[14] Robin Rombach and Patrick Esser. Stable diffusion v1.4 model card, 2022.

[15] Robin Rombach, Andreas Blattmann, Dominik Lorenz, Patrick Esser, and Björn Ommer. High-resolution image synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, 2022. arXiv v2, April 2022; includes supplementary material.

[16] Olaf Ronneberger, Philipp Fischer, and Thomas Brox. U-Net: Convolutional networks for biomedical image segmentation. In *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, pages 234–241, 2015.

[17] Jascha Sohl-Dickstein, Eric Weiss, Niru Maheswaranathan, and Surya Ganguli. Deep unsupervised learning using nonequilibrium thermodynamics. In *Proceedings of the 32nd International Conference on Machine Learning*, pages 2256–2265, 2015.

[18] Jiaming Song, Chenlin Meng, and Stefano Ermon. Denoising diffusion implicit models. In *International Conference on Learning Representations*, 2021.

[19] Yang Song and Stefano Ermon. Generative modeling by estimating gradients of the data distribution. In *Advances in Neural Information Processing Systems*, 2019.

[20] U.S. Copyright Office. Copyright and artificial intelligence, part 3: Generative AI training. Technical report, U.S. Copyright Office, 2025. Pre-publication version; consulted November 5, 2026. A U.S. policy analysis, not a statement of Vietnamese law.

[21] Aäron van den Oord, Nal Kalchbrenner, and Koray Kavukcuoglu. Pixel recurrent neural networks. In *Proceedings of the 33rd International Conference on Machine Learning*, pages 1747–1756, 2016.

[22] Aäron van den Oord, Oriol Vinyals, and Koray Kavukcuoglu. Neural discrete representation learning. In *Advances in Neural Information Processing Systems*, 2017.

[23] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, 2017.

[24] Pascal Vincent. A connection between score matching and denoising autoencoders. *Neural Computation*, 23(7):1661–1674, 2011.

[25] Richard Zhang, Phillip Isola, Alexei A. Efros, Eli Shechtman, and Oliver Wang. The unreasonable effectiveness of deep features as a perceptual metric. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 2018.

A. Requirement-to-Evidence Map

This map concerns the written report. The assignment's separate slide, oral-presentation, balanced-participation, and Q&A requirements require presentation deliverables and cannot be fulfilled by a report alone. The main text occupies pages 1–10; references and appendices are separate.

Table 3. Coverage of the supplied CS420 S06 seminar brief.

Task	Required content	Location / evidence
1	Generative versus discriminative; GAN, autoregressive, diffusion	Sections 2.1, 2.2; Figure 9 (Appendix E)
2	Forward process, beta schedule, Gaussian noise, noise predictor, reverse sampling; both-direction diagram	Section 3; Eqs. (4)–(21); Figure 1
3	Manual 1D forward step, symbols, stochastic expectation	Section 3.4; $x_1 = 0.600833$; conditional mean and variance
4	Small image at five noise levels; optional pretrained demonstration	Section 3.5; Figure 2; executable code and arrays. Figures 6 and 7 show pretrained generation
5	VAE encoder/decoder, latent operations, cross-attention, pipeline	Sections 2.4, 4.2–4.4; Eqs. (1)–(3), (22)–(25); Figure 4
6	Speed, quality, prompt control, data, copyright, bias; explicit pixel/latent comparison	Tables 1 and 2; Figure 5; Sections 5.3, 5.5, and 5.6

Required representations. Equation (4) is the Gaussian forward transition; Eq. (19) is the simplified noise-prediction objective. The worked example gives a complete substitution, result, and interpretation. Appendix C records ranges, units, and modeling assumptions.

Paper reading and synthesis. DDPM's forward/reverse method and contribution are reviewed in Section 3, distinguishing the 2015 foundation from DDPM's parameterization and loss choices. LDM's representation, method, conditioning, contributions, and differences are developed in Section 4. Section 5.4 identifies the published sources for the DDPM objective ablation and LDM inpainting comparison. The report includes Abstract, Introduction, Background/Paper Review, Method/Worked Example, Results and Discussion, Conclusion, and References. English is used according to the commissioning instruction, which overrides the brief's Vietnamese-language request.

B. Reproducibility, Evaluation Protocol, and Attribution

B.1. Reproduce the original illustration

The Overleaf project already contains the generated figure; Python is not needed to compile the report. To regenerate it locally:

```
python -m pip install -r code/requirements.txt
python code/make_figures.py
latexmk -pdf -interaction=nonstopmode -halt-on-
    error main.tex
```

The figure uses a synthetic geometric image, not MNIST or another dataset. The script `code/make_figures.py` simulates one Markov trajectory with NumPy's default generator (seed 420) and the linear schedule $\beta_t = 10^{-4} + (t-1)(0.02 - 10^{-4})/999$. It records the seed, schedule coefficients, library versions, scalar example, and benchmark-ratio arithmetic in `figures/analysis.json`. It saves unclipped frames, timesteps, the schedule, and cumulative coefficients in `noise_arrays.npz`, and summary coefficients in `noise_levels.csv`. The delivered run used NumPy 2.3.5 and ReportLab 4.4.9. The sequence is one sampled Markov path; identical seeds and library versions reproduce it. No image-generation checkpoint is involved.

B.2. Reproduce the Colab runs

The project includes the notebook and run records in `code/colab/`. The LSUN records contain timings, peak allocation, seeds, and checkpoint hashes. The ImageNet record gives the checkpoint URLs, class IDs, and guidance. Its saved trajectories are in Appendix G.

Run `sd14_prompt_control.py` in a Colab GPU cell. The corresponding JSON records the model revision, software versions, scheduler, exact prompts, and initial latent hash for each sample. The visual review CSV records the judgments for all 16 images. A requested color or spatial relation receives credit only when the corresponding vehicle is recognizable. The review marks ambiguous observations separately and skips attributes absent from the baseline.

B.3. Provenance and AI assistance

Figures 1, 4, and 3 are original TikZ schematics explaining cited methods; no paper figure is reproduced. Figure 2 is generated by the supplied code. Tables 1 and 2 and the Colab figures use the saved run records and outputs. Section 5.4 attributes the published DDPM and LDM values to the original papers. Figure 9 is an illustration included with the project, with method sources in its caption.

The document uses the requested ICCV 2025 author kit [9], with its official style and bibliography files. It uses 10-point type, two columns, and US Letter paper, following the new template request. The author block retains HCMUS and CS420 information and three name/ID placeholders. Template provenance is included in the project.

OpenAI Codex [12] assisted with drafting, equations, code, diagram construction, source checking, and LaTeX preparation. Preparation checks included consulting the original DDPM/LDM equations and DDPM Tables 1–2 and LDM Table 6, recalculating the numerical examples and ratios, and compiling and visually reviewing the PDF. The named student authors should review and accept the final manuscript and replace the author placeholders before submission.

C. Notation, Assumptions, and Additional Derivation

C.1. Ranges and units

Images are normalized and dimensionless. Clean pixels in the illustration lie in $[-1, 1]$; noisy states, latent values, and predicted noise are real-valued and need not stay within that interval. There is no physical time unit in the diffusion index.

Table 4. Notation used in the equations; meanings are also introduced beside the corresponding equations in the main text.

Symbols	Meaning, range, and units
$x_0, x_t, \hat{x}_0$	Clean/noisy/reconstructed image; vector in $\mathbb{R}^D$ or an equivalent image tensor; normalized intensity units
D, H, W, h, w, c, d_z	Positive integer dimensions. $D = 3HW$ for RGB, $d_z = hwc$; H, W count pixels, h, w count latent positions
t, T, j	Integer timestep, terminal step, and product index; $1 \le t \le T$, $1 \le j \le t$
$\beta_t, \alpha_t, \bar{\alpha}_t$	Dimensionless schedule coefficients; $0 < \beta_t < 1$, $\alpha_t = 1 - \beta_t$, $0 < \bar{\alpha}_t \le 1$
$q, p_\theta, p_{\text{data}}$	Fixed forward transition, learned reverse/generative distribution, and data distribution, respectively
$\rho, \tilde{x}, q_\rho, s$	Positive Gaussian perturbation SD, perturbed image, noisy marginal, and score in Appendix C.2; s there is a vector field, distinct from the scalar CFG scale
$K_{2015}, \mathcal{D}_t, C_\beta$	Original log-likelihood lower bound, expected posterior KL, and schedule-dependent entropy terms
$\mathcal{L}_{2015}, L_{\text{VLB}}$	Minimized negative 2015 bound and full DDPM variational negative-log-likelihood bound, in nats
L_T, L_{t-1}, L_0	Expected prior KL, intermediate posterior-to-model KL, and final decoder negative log likelihood, in nats
$w_t, \tilde{\beta}_t, \tilde{\mu}_t$	Positive loss weight; forward-posterior variance and mean, for $t > 1$; see Section 3.3
$\mathcal{N}(\mu, \Sigma), \mathbf{I}$	Gaussian with mean μ and covariance Σ; identity covariance of the relevant dimension
$\varepsilon, \varepsilon_z, u, \xi$	Independent standard-normal draws in their respective sampling operations; each coordinate has mean 0 and variance 1
Σ_θ	Learned positive diagonal reverse covariance in the 2015 image model; fixed to $\sigma_t^2 \mathbf{I}$ in the DDPM setup discussed
$\sigma_t, \mu_\phi, \sigma_\phi$	Reverse noise standard deviation $\sigma_t \ge 0$; VAE posterior mean and positive standard deviations
$\theta, \phi, \psi, \eta, \omega$	Trainable parameter arrays of denoiser, VAE encoder, decoder, condition encoder, and illustrative classifier
$\tilde{z}, z_0, z_t, f, a$	Raw/clean-scaled/noisy latent; spatial factor $f \ge 1$, with H, W divisible by f; fixed dimensionless scale $a > 0$
$y, \tau_\eta(y), F, C$	Condition, encoded condition, U-Net features, and token embedding matrix. Continuous embeddings are real-valued
n, m, d_u, d_y, d_a, d_v	Positive integer counts and widths for cross-attention; see Appendix D.1 for all matrix shapes
Q, K, V, A, O	Query, key, value, attention-weight, and output matrices; $A_{ij} \in [0, 1]$ and $\sum_j A_{ij} = 1$
$s, w, p_{\text{drop}}, \varnothing$	CFG scale $s \ge 0$, alternative scale $w = s - 1$, dropout probability in $(0, 1)$, and null condition
$K, r, \mathcal{T}$	In the cost model only, K counts updates (not attention keys), r counts evaluations/update, and $\mathcal{T}$ is seconds
$L_{\text{simple}}, L_{\text{LDM}}$	Expected summed squared noise-prediction errors; dimensionless squared normalized-coordinate units
$\mathcal{J}_{\text{VAE}}$	Textbook VAE negative ELBO, in nats
$\mathcal{E}_\phi, D_\psi, \mathcal{Z}$	LDM encoder, decoder, and VQ codebook $\mathcal{Z} \subset \mathbb{R}^c$ used by the decoder's quantization layer
$\mathbb{E}, \lVert \cdot \rVert_2, \mathrm{D}_{\text{KL}}$	Expectation, Euclidean norm, and KL divergence; natural logarithms make KL and negative log likelihood use nats

C.2. Optional background: scores and Gaussian perturbation

For a smooth positive density p, define its score as

$$s(x) = \nabla_x \log p(x). \tag{28}$$

This vector gives the direction of steepest local increase in log density, not a scalar image-quality rating. For $p(x) = \mathcal{N}(0,1)$, $\log p(x) = -x^2/2 + C$ and $s(x) = -x$. Score-based sampling combines density-directed drift with randomness. For example, a discretized Langevin update is [19]

$$v^{(k+1)} = v^{(k)} + \frac{\delta}{2} s(v^{(k)}) + \sqrt{\delta}\,\zeta^{(k)}. \tag{29}$$

Here $v^{(k)}$ is the current state, k counts updates, $\delta > 0$ is a step size, and $\zeta^{(k)} \sim \mathcal{N}(0, \mathbf{I})$ are independent draws. The stochastic term supports distributional exploration rather than only mode seeking. Finite-step sampling is approximate; convergence requires suitable step-size limits and regularity conditions. This supplies context for the score-based literature rather than an additional algorithm required by the DDPM construction in the main text.

Gaussian smoothing addresses difficulties in estimating scores when data concentrate near low-dimensional subsets of image space [19]. A simple additive example is

$$\tilde{x} = x_0 + \rho\varepsilon, \quad x_0 \sim p_{\text{data}}, \quad \varepsilon \sim \mathcal{N}(0, \mathbf{I}), \quad \rho > 0. \tag{30}$$

ρ is a noise standard deviation. Its known Gaussian conditional $q_\rho(\tilde{x} \mid x_0) = \mathcal{N}(x_0, \rho^2\mathbf{I})$ has score $-\varepsilon/\rho$. The marginal $q_\rho(\tilde{x})$ averages over clean data; learning its score requires the conditional-to-marginal identity below. Different noise levels define different smoothed distributions. DDPM instead scales the clean signal as well, as in Eq. (5).

C.3. Why the forward marginal is available in closed form

Assume independent standard-normal innovations $\varepsilon^{(1)}, \varepsilon^{(2)}$ and let x_0 be fixed. Applying Eq. (4) twice gives

$$\begin{aligned} x_1 &= \sqrt{\alpha_1}x_0 + \sqrt{\beta_1}\varepsilon^{(1)}, \\ x_2 &= \sqrt{\alpha_2\alpha_1}x_0 + \sqrt{\alpha_2\beta_1}\varepsilon^{(1)} + \sqrt{\beta_2}\varepsilon^{(2)}. \end{aligned}$$

The two independent noise terms have total variance $\alpha_2\beta_1 + \beta_2 = \alpha_2(1-\alpha_1) + (1-\alpha_2) = 1 - \alpha_2\alpha_1$. A sum of independent Gaussians is Gaussian. Repeating this argument yields Eq. (5), with mean $\sqrt{\bar{\alpha}_t}x_0$ and covariance $(1-\bar{\alpha}_t)\mathbf{I}$ [8].

Gaussian innovations are independent across coordinates and steps, but the noisy images themselves have correlations inherited from x_0. When $\bar{\alpha}_T$ is close to zero, dependence on the starting image is weak and a standard-normal starting prior is a useful approximation. The learned Gaussian reverse transition is a modeling approximation motivated by small forward steps; no finite network is assumed to know the exact reverse conditional.

C.4. Denoising score matching and the MSE optimum

Fix a nonzero noise level and write $x_t = a_t x_0 + b_t\varepsilon$ with $a_t, b_t > 0$, $\varepsilon \sim \mathcal{N}(0, \mathbf{I})$ independent of x_0. The additive example of Appendix C.2 uses $a_t = 1, b_t = \rho$; DDPM uses $a_t = \sqrt{\bar{\alpha}_t}$ and $b_t = \sqrt{1-\bar{\alpha}_t}$. These a_t coefficients are unrelated to the fixed latent scale a. Let q_t be the marginal over noisy data. Gaussian smoothing makes it positive and smooth; assume sufficient integrability to exchange integration and differentiation. Then

$$\begin{aligned} q_t(x_t) &= \int p_{\text{data}}(x_0) q(x_t \mid x_0)\, dx_0, \\ \nabla \log q_t(x_t) &= \int q(x_0 \mid x_t, t) \nabla \log q(x_t \mid x_0)\, dx_0. \end{aligned} \tag{31}$$

The conditional Gaussian has score

$$\nabla \log q(x_t \mid x_0) = -\frac{x_t - a_t x_0}{b_t^2} = -\frac{\varepsilon}{b_t}. \tag{32}$$

Taking its posterior expectation in Eq. (31) gives

$$\nabla \log q_t(x_t) = -\frac{\mathbb{E}[\varepsilon \mid x_t, t]}{b_t}. \tag{33}$$

This is the conditional-to-marginal identity underlying denoising score matching [19, 24].

For any vector-valued predictor g and finite second-moment target Y, condition on an input X. With $m(X) = \mathbb{E}[Y \mid X]$, expand

$$\begin{aligned} \mathbb{E}[\|Y - g(X)\|^2 \mid X] = \mathbb{E}[\|Y - m(X)\|^2 \mid X] \\ + \|m(X) - g(X)\|^2. \end{aligned} \tag{34}$$

The cross term vanishes because $\mathbb{E}[Y - m(X) \mid X] = 0$. Thus the ideal MSE predictor is $m(X)$. Setting $Y = \varepsilon$ and $X = (x_t, t)$ justifies the interpretation in Section 3.3. A finite trained network only approximates that optimum.

If $\varepsilon_\theta = -b_t s_\theta$, a conditional-target score loss satisfies

$$\mathbb{E}\|s_\theta + \varepsilon/b_t\|^2 = \frac{1}{b_t^2}\mathbb{E}\|\varepsilon - \varepsilon_\theta\|^2 \tag{35}$$

at a fixed t. Across timesteps, weights must be accounted for: unweighted noise MSE corresponds to weighting this score loss by b_t^2. The two unweighted losses should not be identified without that factor. The identity is used only for $b_t > 0$, not at the clean endpoint.

C.5. The original 2015 bound and learned covariance

For a Gaussian diffusion model, the general trajectory lower bound is

$$\mathbb{E}[\log p_\theta(x_0)] \geq \mathbb{E}_q[\log p_\theta(x_{0:T}) - \log q(x_{1:T} \mid x_0)]. \tag{36}$$

Sohl-Dickstein et al. [17, Sec. 2.4, App. B] rewrite this, under their normalization and stationary-kernel endpoint conventions, as

$$\begin{aligned} K_{2015} &= -\sum_{t=2}^{T} \mathcal{D}_t + C_\beta, \\ \mathcal{D}_t &= \mathbb{E}_q\left[\mathrm{D}_{\mathrm{KL}}\left(q(x_{t-1} \mid x_t, x_0) \| p_\theta(x_{t-1} \mid x_t)\right)\right], \\ C_\beta &= H_q(X_T \mid X_0) - H_q(X_1 \mid X_0) - H_\pi(X_T). \end{aligned} \tag{37}$$

The minimized 2015 loss is $\mathcal{L}_{2015} = -K_{2015}$. H denotes differential entropy in nats and $\pi = \mathcal{N}(0, \mathbf{I})$. The stationary-prior cross-entropy simplification uses the paper's normalization of the data. For the isotropic Gaussian forward chain, $H_q(X_t \mid X_0) = \frac{D}{2}\log[2\pi e(1-\bar{\alpha}_t)]$. These terms depend on the schedule; treating C_β as constant is valid with respect to reverse parameters at a fixed schedule, not when optimizing the forward rates.

In particular, the 2015 endpoint is prescribed as

$$p(x_0 \mid x_1) = T_\pi(x_0 \mid x_1; \beta_1) = \mathcal{N}(\sqrt{1-\beta_1}x_1, \beta_1 \mathbf{I}), \tag{38}$$

where T_π is the stationary Gaussian kernel. This removes the learned endpoint term from its KL sum. It should not be confused with DDPM's endpoint likelihood convention or its final noiseless display update.

For $q = \mathcal{N}(\tilde{\mu}_t, \tilde{\beta}_t \mathbf{I})$ and learned diagonal $p_\theta = \mathcal{N}(\mu_\theta, \mathrm{diag}(v_{\theta,1}, \ldots, v_{\theta,D}))$, with $v_{\theta,i} > 0$, the per-example KL is

$$\begin{aligned} \mathrm{D}_{\mathrm{KL}}(q\|p_\theta) = \frac{1}{2}\sum_{i=1}^{D}\Bigg[\log\frac{v_{\theta,i}}{\tilde{\beta}_t} - 1 \\ + \frac{\tilde{\beta}_t + (\tilde{\mu}_{t,i} - \mu_{\theta,i})^2}{v_{\theta,i}}\Bigg]. \end{aligned} \tag{39}$$

Learning covariance therefore changes more than a mean-MSE target. Only after fixing reverse variance do the variance-dependent pieces become constant in θ, giving the mean-MSE expression used in the parameterization argument in Section 3.3.

C.6. Why the score determines the reverse mean

Apply the marginal-score identity to a single Gaussian forward step:

$$\nabla \log q_t(x_t) = -\frac{x_t - \sqrt{\alpha_t}\mathbb{E}[x_{t-1} \mid x_t, t]}{\beta_t}. \tag{40}$$

Therefore its true reverse conditional mean is

$$\mathbb{E}[x_{t-1} \mid x_t, t] = \frac{x_t + \beta_t \nabla \log q_t(x_t)}{\sqrt{\alpha_t}}. \tag{41}$$

Using the estimated DDPM score $-\varepsilon_\theta(x_t, t)/\sqrt{1-\bar{\alpha}_t}$ gives the reverse mean in Eq. (12). The conditional-mean identity is exact under the stated assumptions, but it does not make the full reverse conditional Gaussian. Its higher moments and possible multimodality are not specified by the mean alone. DDPM uses a Gaussian approximation motivated by small forward steps and learns an approximate score.

C.7. Why noise prediction is useful without knowing the true noise

For a fixed x_t and t, the vector minimizing expected squared prediction error is $\mathbb{E}[\varepsilon \mid x_t, t]$ (and, in a conditional model, $\mathbb{E}[\varepsilon \mid x_t, t, y]$). Thus a trained predictor estimates a statistical direction shared by plausible clean examples, not an oracle that recovers every random draw. Its corresponding clean-state estimate is

$$\hat{x}_0(x_t, t) = \frac{x_t - \sqrt{1-\bar{\alpha}_t}\varepsilon_\theta(x_t, t)}{\sqrt{\bar{\alpha}_t}}. \tag{42}$$

This follows by rearranging Eq. (5); prediction errors remain and can be magnified at small $\bar{\alpha}_t$. The reverse sampler uses predictions over many levels rather than treating one estimate as guaranteed recovery. The same argument applies in latent space, followed by the learned decoder.

D. Supporting Conditioning Details

D.1. Cross-attention matrix shapes

This subsection gives the shapes behind Eqs. (26)–(27), using a *row-vector* convention. The flattened feature map $F \in \mathbb{R}^{n \times d_u}$ has n spatial positions of channel width d_u; $C = \tau_\eta(y) \in \mathbb{R}^{m \times d_y}$ contains m token embeddings of width d_y. The learned projections have shapes $W_Q \in \mathbb{R}^{d_u \times d_a}$, $W_K \in \mathbb{R}^{d_y \times d_a}$, and $W_V \in \mathbb{R}^{d_y \times d_v}$. Thus $Q \in \mathbb{R}^{n \times d_a}$ contains *queries from image features*, $K \in \mathbb{R}^{m \times d_a}$ contains *keys from text*, and $V \in \mathbb{R}^{m \times d_v}$ contains text values. d_a, d_v are positive integer head widths. A superscript $\top$ denotes transposition. Softmax acts across the m tokens for each spatial row, making $A \in \mathbb{R}^{n \times m}$ nonnegative with row sums of one. $O \in \mathbb{R}^{n \times d_v}$ supplies text-dependent features that are projected back into the U-Net's residual stream.

A numerical attention example

Take one spatial query $Q = (1, 0)$, two keys $K_1 = (\sqrt{2}, 0)$, $K_2 = (0, 0)$, and $d_a = d_v = 2$. The scaled logits are $(1, 0)$, so $A = (e/(e+1), 1/(e+1)) \approx (0.7311, 0.2689)$, where e is Euler's number. For $V_1 = (1, 0)$ and $V_2 = (0, 1)$, the output is $O \approx (0.7311, 0.2689)$. This original toy example shows weighted retrieval, not an observed attention map from an actual model.

D.2. Prompt control and classifier-free guidance

Cross-attention supplies the condition; *classifier-free guidance* (CFG) adjusts how strongly a conditional prediction influences sampling [7]. Train the same network with the condition replaced by a null condition $\varnothing$ with probability $p_{\text{drop}} \in (0, 1)$. At inference define

$$\begin{aligned} \varepsilon_u &= \varepsilon_\theta(z_t, t, \tau_\eta(\varnothing)), \\ \varepsilon_c &= \varepsilon_\theta(z_t, t, \tau_\eta(y)), \\ \varepsilon_{\text{cfg}} &= \varepsilon_u + s(\varepsilon_c - \varepsilon_u). \end{aligned} \tag{43}$$

$s \geq 0$ is a dimensionless guidance scale. Here $s = 0$ uses the unconditional prediction, $s = 1$ gives ordinary conditional sampling, and $s > 1$ extrapolates in the conditional direction. The CFG paper writes the equivalent expression $(1 + w)\varepsilon_c - w\varepsilon_u$; its parameter is $w = s - 1$. Stating the convention prevents confusing two scales with different numerical baselines.

For an original scalar illustration, $\varepsilon_u = 0.2$, $\varepsilon_c = 0.1$ and $s = 3$ give $\varepsilon_{\text{cfg}} = -0.1$. Guidance is therefore an extrapolation, not a convex average. Substituting it into the reverse update changes the generation trajectory. Greater guidance can improve conditional fidelity while reducing diversity; it does not ensure monotonically better quality [7]. Standard CFG requires two denoiser evaluations per step, often batched together, adding compute and potentially peak memory.

Our prompt-control test with fixed seeds and separate attribute and relation edits is reported in Section 5.5 and Appendix H.

E. Qualitative Comparison of Generative Models

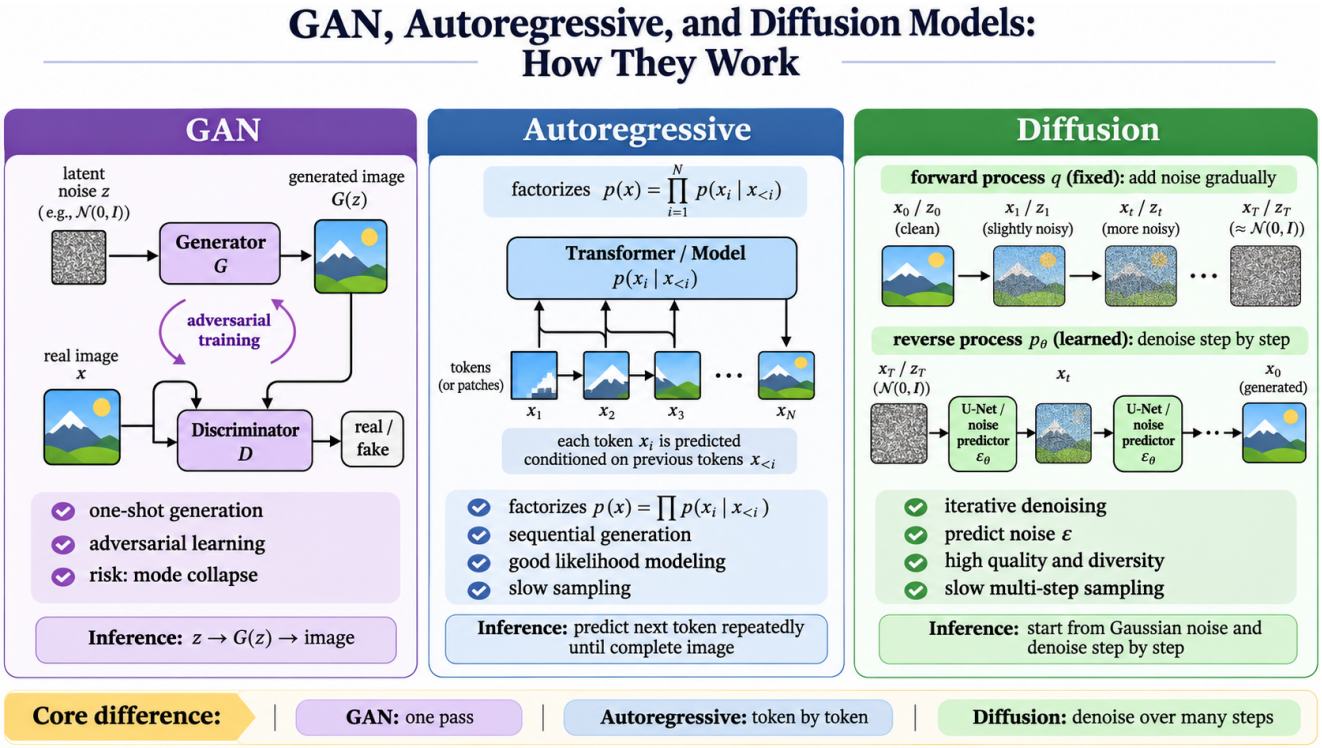


Figure 9. Overview of GAN, autoregressive, and diffusion models. Related methods are described in [5, 8, 21]. The labels describe typical behavior rather than universal performance guarantees.

F. Pseudocode for Forward and Backward Flows

These algorithms make the Gaussian processes in Sections 3.1–3.2 explicit, using the 2015 reverse-parameter formulation [17]. All states lie in $\mathbb{R}^D$; each Gaussian noise draw is independent. The two procedures describe corruption and generation; the DDPM variant is distinguished below.

F.1. Forward noising

Algorithm 1 Forward Gaussian diffusion

Input: Clean state x_0; schedule $\beta_1, \dots, \beta_T$
Output: Forward trajectory $(x_0, x_1, \dots, x_T)$

for $t = 1, \dots, T$ **do**
  Draw $\varepsilon^{(t)} \sim \mathcal{N}(0, \mathbf{I})$
  $\alpha_t \leftarrow 1 - \beta_t$
  $x_t \leftarrow \sqrt{\alpha_t}\, x_{t-1} + \sqrt{\beta_t}\, \varepsilon^{(t)}$
end for
return $(x_0, x_1, \dots, x_T)$

What this computes. Algorithm 1 applies the known Gaussian transition in Eq. (4). The schedule satisfies $0 < \beta_t < 1$; the original Gaussian framework may learn its rates, whereas the reported DDPM implementation fixes them. The forward transition itself does not evaluate a denoising network.

Direct access to one timestep. When only one noisy training input is required, the full loop is unnecessary. Compute $\bar{\alpha}_t = \prod_{j=1}^{t}(1 - \beta_j)$ and draw

$$x_t = \sqrt{\bar{\alpha}_t}\, x_0 + \sqrt{1 - \bar{\alpha}_t}\, \varepsilon, \qquad \varepsilon \sim \mathcal{N}(0, \mathbf{I}).$$

This samples the same marginal $q(x_t \mid x_0)$ as Eq. (5); it does not recover the particular path generated by Algorithm 1.

Separate runs and separate noise. Backward generation starts from a fresh prior draw, independently of the clean image and trajectory used in Algorithm 1. The reverse noise u_t is not the forward innovation $\varepsilon^{(t)}$. Composing the learned reverse transitions produces a model sample; it does not guarantee recovery of the original x_0.

F.2. Backward sampling: the 2015 Gaussian model

Algorithm 2 Backward Gaussian sampling

Input: T; trained f_μ, f_Σ; first-step variance β_1
Output: A generated state x_0

Draw $x_T \sim \mathcal{N}(0, \mathbf{I})$
for $t = T, T-1, \dots, 2$ **do**
  $\mu_t \leftarrow f_\mu(x_t, t)$
  $\Sigma_t \leftarrow f_\Sigma(x_t, t)$
  Draw $u_t \sim \mathcal{N}(0, \mathbf{I})$
  $x_{t-1} \leftarrow \mu_t + \Sigma_t^{1/2} u_t$
end for
Draw $u_1 \sim \mathcal{N}(0, \mathbf{I})$
$x_0 \leftarrow \sqrt{1 - \beta_1}\, x_1 + \sqrt{\beta_1}\, u_1$
return x_0

Learned intermediate steps. The network receives the current noisy state and timestep, and predicts a reverse mean and positive covariance. For the original image model, Σ_t is diagonal and its square root is computed elementwise. The random draw implements $p_\theta(x_{t-1} \mid x_t) = \mathcal{N}(\mu_t, \Sigma_t)$, as in Eq. (7).

The 2015 endpoint. The last update uses the prescribed stationary Gaussian kernel [17, App. B.2], rather than another learned transition. Its formula and role in the original variational bound are given in Appendix C.5.

How DDPM changes the backward sampler. DDPM [8, Algorithm 2] computes μ_θ from ε_θ using Eq. (12) and uses a fixed $\sigma_t^2 \mathbf{I}$ for the intermediate reverse covariance. The two choices, $\sigma_t^2 = \beta_t$ and $\sigma_t^2 = \tilde{\beta}_t$, are specified in Eq. (20). Its update is $x_{t-1} = \mu_\theta(x_t, t) + \sigma_t u_t$ for $t = T, \dots, 1$, drawing $u_t \sim \mathcal{N}(0, \mathbf{I})$ when $t > 1$ and setting $u_1 = 0$ for the final update. Thus the 2015 endpoint in Algorithm 2 must also be replaced when implementing DDPM. Forward noising remains Algorithm 1; Section 3.3 explains the parameterization and sampling changes.

G. ImageNet Forward and Reverse Trajectories

Each panel follows one class for both models. Read each reverse row from the initial noise through updates $10, 20, 30, 40$ to the final image at update 50. Each forward row starts from that final state and shows noising at $t = 0, 50, 150, 300, 600, 1000$. ADM states are in pixel space. LDM states are decoded from latent space for display.

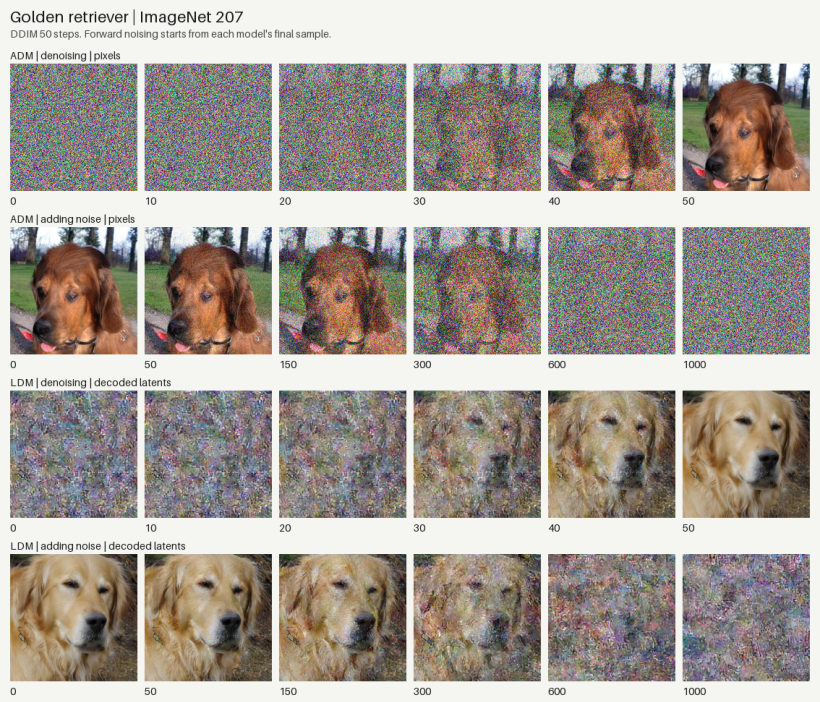


Figure 10. Golden retriever (ImageNet class 207).

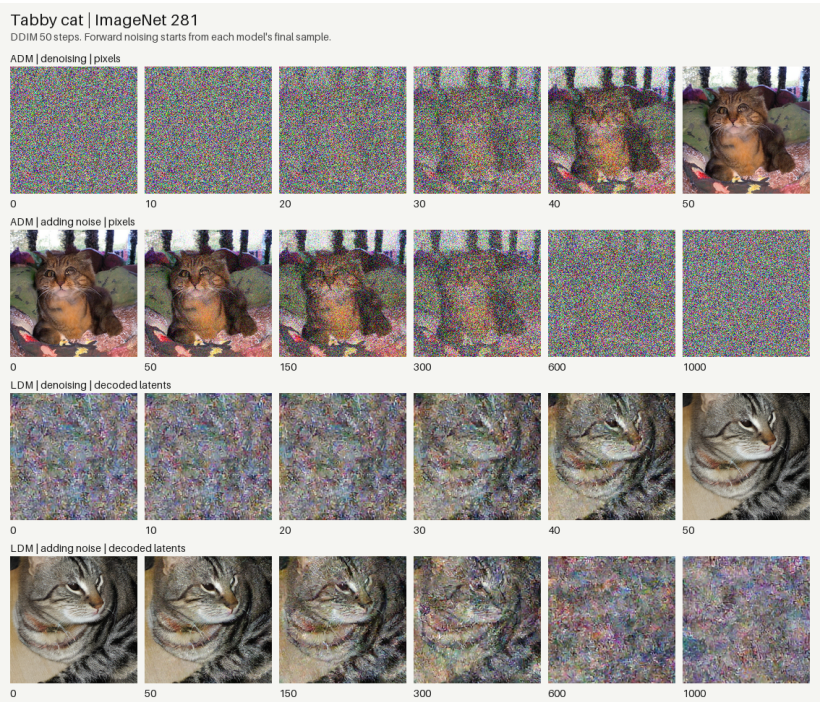


Figure 11. Tabby cat (ImageNet class 281).

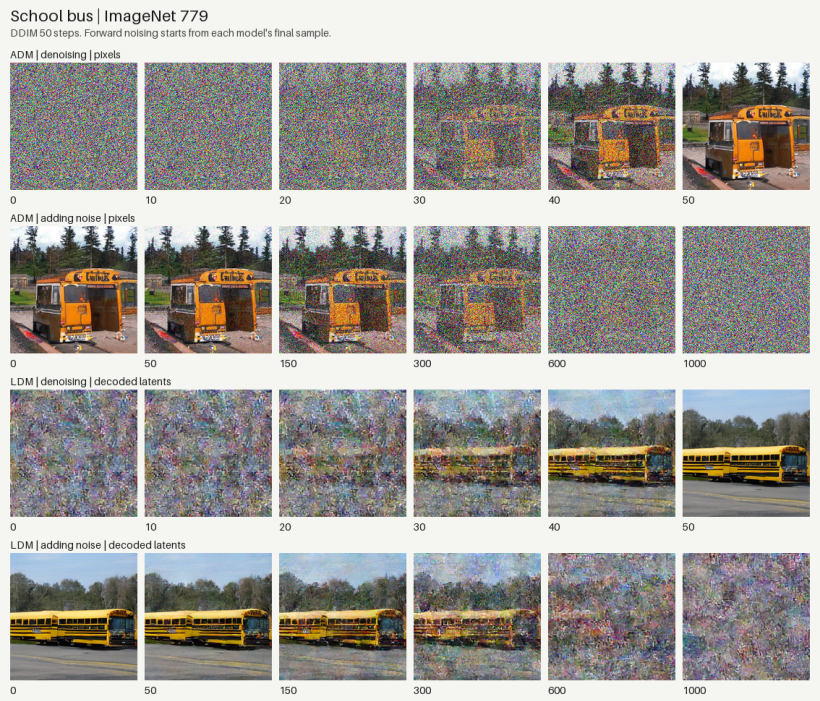


Figure 12. School bus (ImageNet class 779).

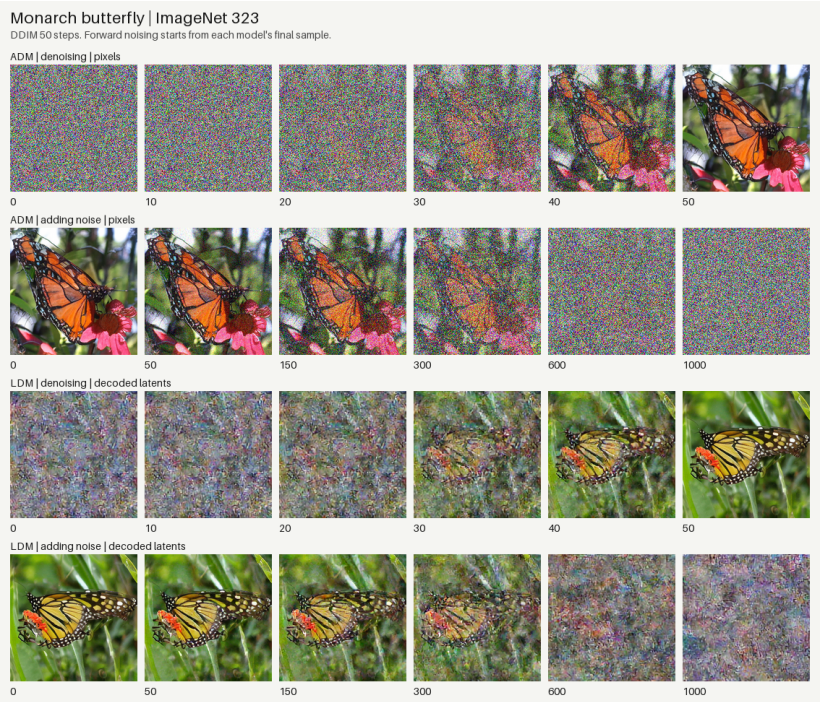


Figure 13. Monarch butterfly (ImageNet class 323).

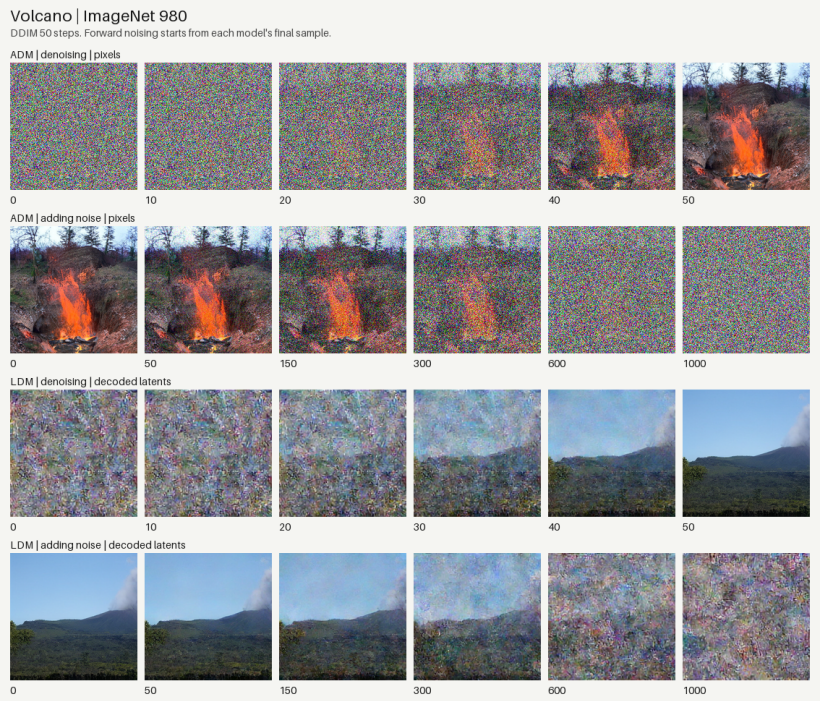


Figure 14. Volcano (ImageNet class 980).

H. Full Text Prompt Comparison

Each row reuses its initial latent across the four prompts. The reference specifies colors and a spatial relation. The two edit columns each change one word in that reference. All seeds are shown.

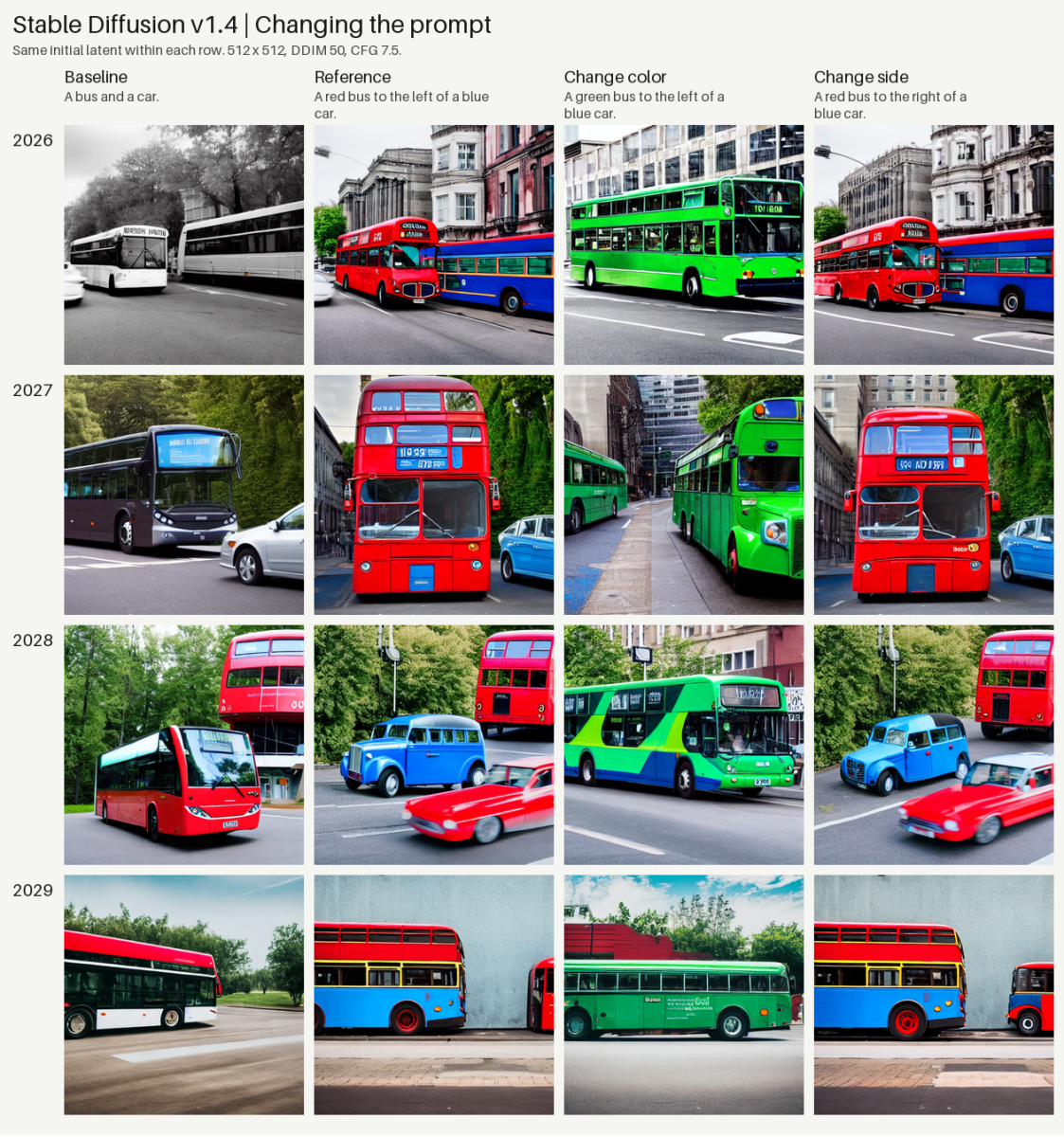


Figure 15. All 16 Stable Diffusion v1.4 samples from the supplementary test in Section 5.5.